

CKBuilder Program

Week 2 Journal

Project: CKB Profile Cell Validator

Author: Jimleston Osoi

Program: CKBuilder Program – Nervos CKB Developer Track

Overview

This week I transitioned from studying the fundamentals of Nervos CKB to implementing a practical Type Script in Rust. The project focuses on validating a profile Cell stored on-chain and enforcing rules on how that Cell can be created and updated.

Objectives

- Understand CKB Type Scripts
- Read Cell data using `ckb-std`
- Validate transaction outputs
- Return meaningful error codes
- Write unit tests
- Organize the project using the standard CKB contract structure

Implementation

The contract validates a profile Cell whose data follows:

```
```text
```

```
CKB_PROFILE_V1|name=Jimleston_Osoi
```

```
```
```

Validation rules:

1. Exactly one profile output must exist.
2. A profile Cell cannot be destroyed.
3. The profile must begin with `CKB\_PROFILE\_V1|name=`.
4. The name cannot be empty.
5. The data must not exceed 128 bytes.
6. Only letters, numbers, spaces, `\_`, and `-` are allowed.

Repository Updates

- Added `week-02-profile-validator`
- Added validation logic
- Added custom error codes
- Added unit tests
- Updated build configuration

- Documented the implementation

Challenges

- Understanding ``Source::GroupInput`` and ``Source::GroupOutput``
- Structuring validation logic
- Configuring the RISC-V toolchain for CKB

Key Learnings

- CKB validates transactions through scripts rather than modifying account state.
- Type Scripts enforce application-specific rules.
- Cells are immutable; updates create new Cells.

Next Week

- Use script arguments
- Explore Type ID
- Build more advanced validation logic